

LOW-RESOURCE ASR WITH PRETRAINED SPEECH SELF-SUPERVISED REPRESENTATIONS

Yuhua Luo

524531910035

Haomin Hu

524531910040

Yuecheng Han

524531910029

School of Artificial Intelligence
Shanghai Jiao Tong University
Shanghai, China

ABSTRACT

This project studies low-resource automatic speech recognition (ASR) with pretrained speech self-supervised learning (SSL) representations. We implement a LibriSpeech pipeline with transcript normalization, JSONL manifests, character-level CTC training, greedy decoding, WER/CER evaluation, and qualitative error analysis. Using approximately one hour from LibriSpeech train-clean-100, we compare a small log-mel CTC baseline with wav2vec 2.0 and HuBERT encoders followed by a lightweight CTC head. A five-layer sweep across wav2vec 2.0 reveals a clear U-shaped curve: layer 9 achieves 0.617 test WER, substantially outperforming both the shallowest (layer 0, 0.974 WER) and deepest (layer 12, 0.986 WER) representations. We further conduct two-phase fine-tuning experiments, partially unfreezing the top six transformer layers after a frozen warm-up. HuBERT fine-tuning yields the best overall result (0.320 test WER, 0.106 test CER), halving the error rate of its frozen counterpart. In contrast, wav2vec 2.0 fine-tuning provides no improvement over the best frozen layer, suggesting that representation selection can outweigh fine-tuning when labeled data are extremely scarce. These findings confirm that pretrained SSL representations are useful in low-resource ASR, and that both hidden layer choice and pretraining objective critically shape downstream performance.

Index Terms— Low-resource ASR, self-supervised learning, wav2vec 2.0, HuBERT, CTC

1. INTRODUCTION

Low-resource automatic speech recognition (ASR) is difficult because the model must learn an acoustic-to-text mapping from limited paired speech and transcripts. In conventional supervised ASR, thousands of hours of labeled speech are used to train end-to-end models [?]. When only a small amount of labeled data is available—one hour or less—the model lacks sufficient supervision

to learn robust acoustic-phonetic mappings, leading to poor generalization.

Speech self-supervised learning (SSL) offers a way to reduce this dependence on labels. Large unlabeled audio corpora can pretrain acoustic encoders, after which a low-resource system can train a small supervised decoder on top of their hidden states.

This project follows that paradigm and builds an English ASR system centered on pretrained wav2vec 2.0 and HuBERT representations. The study asks three main questions: (1) Do pretrained SSL hidden states help in a one-GPU, low-resource LibriSpeech setting where only one hour of labeled speech is available? (2) Does the choice of SSL model and hidden layer matter when the downstream model is deliberately small and the encoder is kept frozen? (3) Can partial encoder fine-tuning substantially improve results, and does its effectiveness depend on the pretraining objective?

Our contributions are: (i) a reproducible end-to-end low-resource ASR pipeline, (ii) a controlled comparison of wav2vec 2.0 and HuBERT against a deliberately small non-pretrained log-mel control, (iii) a five-layer sweep across wav2vec 2.0 demonstrating a clear U-shaped WER curve and identifying layer 9 as the optimal frozen representation, and (iv) two-phase fine-tuning experiments revealing that HuBERT benefits dramatically from partial unfreezing (WER halved from 0.663 to 0.320) while wav2vec 2.0 does not, highlighting a critical interaction between pretraining objective and low-resource fine-tuning. All experiments run on a single consumer GPU (RTX 4060, 8 GB VRAM), making the setup accessible and reproducible.

2. RELATED WORK

LibriSpeech is a standard English read-speech benchmark for ASR research and provides the clean speech splits used in this study [?]. Self-supervised speech encoders such as wav2vec 2.0 [?] and HuBERT [?] learn

representations from raw audio without transcripts, using contrastive or masked-prediction objectives. Benchmarks such as SUPERB show that frozen SSL representations transfer to downstream speech tasks including phoneme recognition, keyword spotting, and ASR [?].

CTC [?] is widely used for end-to-end ASR because it trains sequence models without frame-level alignments. This project uses character-level CTC with greedy decoding and no external language model, lexicon, or beam search. That choice makes absolute WER higher than a full production ASR system, but keeps the comparison focused on the acoustic representations themselves.

Layer-wise analyses of SSL models show that intermediate layers often encode phonetic and articulatory information more explicitly, while final layers become more specialized to the pretraining objective [?]. This motivates the main ablation in this paper: rather than ranking many ASR systems, we isolate how the hidden-state choice affects a small frozen-encoder CTC model trained from one hour of labeled speech.

3. METHOD

3.1. System Overview

The overall system architecture is shown in Fig. 1. The pipeline takes raw 16 kHz audio waveforms, passes them through a frozen SSL encoder to extract frame-level hidden states, projects these to character logits via a lightweight CTC head, and applies greedy CTC decoding to produce the transcript. For the log-mel baseline, the SSL encoder is replaced with a small feed-forward network over 80-dimensional log-mel filterbank features.

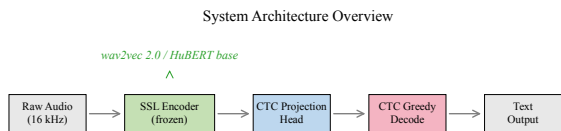


Fig. 1. System architecture. Raw audio passes through a frozen SSL encoder (wav2vec 2.0 or HuBERT base), a CTC projection head, and greedy CTC decoding.

3.2. Data and Normalization

All experiments use LibriSpeech [?], a public-domain English read-speech corpus. To avoid unreliable long downloads from OpenSLR, the data preparation script reads the Hugging Face Parquet mirror (openslr/librispeech_asr), materializes FLAC files locally, and writes JSONL manifests. The training split contains 269 utterances totaling

1.004 hours, sampled from train-clean-100. The development and test manifests contain 2703 and 2620 utterances from dev-clean and test-clean, respectively.

Transcripts are lowercased, punctuation is removed, apostrophes are preserved, and whitespace is collapsed. The CTC vocabulary is character-level with 30 tokens: blank, pad, unk, 26 letters a–z, apostrophe, and space. Vocabulary is built from the training split only; no external lexicon or language model is used.

3.3. Log-Mel CTC Baseline

The baseline extracts 80-dimensional log-mel filterbank features (25 ms Hamming window, 10 ms shift) via torchaudio, applies $\log(1+x)$ compression, and feeds the sequence to a feed-forward encoder with four hidden layers (256 units each, ReLU, dropout 0.1). A linear CTC head projects to character logits. This model has ~ 0.33 M trainable parameters and contains no pretrained speech knowledge; it serves as the non-SSL control.

3.4. SSL CTC Models

The SSL systems use Hugging Face Transformers encoders with a compact CTC projection head. We evaluate two pretrained models: facebook/wav2vec2-base (95.6 M parameters) [?] and facebook/hubert-base-ls960 (95.0 M parameters) [?]. Both accept 16 kHz raw waveforms and output 768-dimensional hidden states with a 20 ms frame interval ($320\times$ CNN subsampling). The CTC projection head is $\text{Linear}(768 \rightarrow 768) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.1) \rightarrow \text{ear}(768 \rightarrow V)$, where V is the vocabulary size.

In all SSL runs, the encoder is frozen and only the projection head is trained (~ 1.2 M trainable parameters). This keeps the experiment feasible on an 8 GB GPU and isolates the value of pretrained representations.

For layer analysis, we sweep five wav2vec 2.0 hidden layers (0, 3, 6, 9, and 12/final), holding all other variables—dataset, optimizer, frozen encoder, CTC head, and decoding procedure—constant. This is a single-variable ablation designed to map the full depth-wise trend rather than comparing only two checkpoints.

3.5. Two-Phase Fine-tuning

To investigate whether partial encoder fine-tuning can improve results under the low-resource constraint, we conduct two-phase training for wav2vec 2.0 layer 6 and HuBERT last. Phase A (epochs 1–3) keeps the encoder fully frozen and trains only the CTC projection head at $\text{lr} = 10^{-3}$, identical to the frozen-only experiments. Phase B (epochs 4–8) unfreezes the top six transformer layers and continues training at a reduced learning rate of 10^{-4} . The optimizer is re-initialized at the phase

boundary to avoid momentum carryover from the frozen phase. This design isolates the marginal benefit of fine-tuning while keeping total training time manageable on a single consumer GPU.

3.6. Experimental Controls

All systems share the same text normalization, character vocabulary, CTC objective [?], and greedy CTC decoding. No external language model, beam search, or weight averaging is used. This makes the comparison conservative: an LM would improve absolute WER but would obscure how much information the acoustic representation itself contributes. The log-mel system is the non-pretrained control; the frozen HuBERT and wav2vec 2.0 systems test the effect of SSL pretraining; the five-layer wav2vec 2.0 sweep isolates the hidden-layer choice; and the two-phase fine-tuning runs examine whether the benefit of pretrained representations can be amplified through limited supervised adaptation.

4. EXPERIMENTS

4.1. Setup and Metrics

All training used PyTorch 2.7.1 with CUDA 12.8 on an NVIDIA GeForce RTX 4060 Laptop GPU (8 GB VRAM). Training hyperparameters are summarized in Table 1. All runs used the AdamW optimizer, gradient clipping at 1.0, and fixed random seed 1337 with no stochastic data augmentation.

Table 1. Training hyperparameters. FT = two-phase fine-tuning (3 frozen epochs + 5 fine-tune epochs).

	Log-mel	wav2vec2 frozen	wav2vec2 FT	HuBERT frozen	HuBERT FT
Epochs	8	5	3+5	5	3+5
Batch size	8	2	2	2	2
Learning rate	3×10^{-4}	10^{-3}	$10^{-3} / 10^{-4}$	10^{-3}	$10^{-3} / 10^{-4}$
Gradient clip	1.0	1.0	1.0	1.0	1.0
Max train audio	18s	18s	18s	18s	18s
Max eval audio	30s	30s	30s	30s	30s
Trainable params	0.33M	1.2M	~42M	1.2M	~42M

WER and CER, computed via Levenshtein alignment at the word and character level, are the primary metrics. Real-time factor—inference wall time divided by audio duration—is a secondary efficiency metric. For SSL models, evaluation uses a 30-second maximum utterance duration, covering 2694/2703 dev and 2611/2620 test utterances. The log-mel run attains WER = CER = 1.000 under both 30-second and 60-second limits, so the exclusion does not affect any conclusion. All reported dev/test metrics are computed on these duration-filtered subsets unless otherwise stated.

Table 2. Low-resource ASR performance using 1 hour of LibriSpeech train-clean-100.

System	Dev WER	Test WER	Test CER
HuBERT last (FT top-6)	0.316	0.320	0.106
HuBERT last	0.656	0.663	0.232
Log-mel	1.000	1.000	1.000
wav2vec2 layer 6 (FT top-6)	0.678	0.683	0.271
wav2vec2 last	0.989	0.986	0.640
wav2vec2 layer 0	0.975	0.974	0.582
wav2vec2 layer 3	0.913	0.915	0.418
wav2vec2 layer 6	0.666	0.670	0.281
wav2vec2 layer 9	0.605	0.617	0.243

4.2. Main Results

Table ?? presents the primary results. Fig. 2 shows training dynamics, and Fig. 3 provides a visual comparison of test-set error rates.

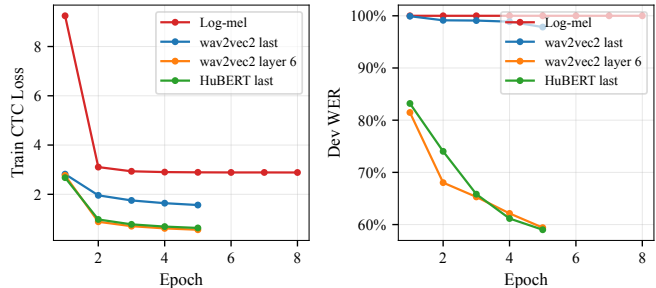


Fig. 2. Training dynamics. (Left) CTC training loss per epoch. (Right) Development WER per epoch. HuBERT and wav2vec 2.0 layer 6 converge rapidly and reach substantially lower dev WER.

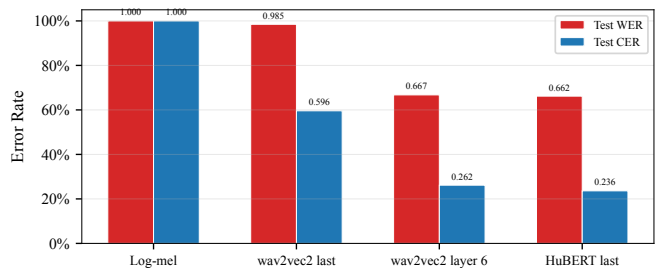


Fig. 3. Duration-filtered test-set WER and CER across all systems.

The log-mel baseline is included as a deliberately small, non-pretrained control. It converges to empty or near-empty hypotheses for every utterance, yielding 1.000 WER and CER. This is the CTC blank-dominance

failure mode: with one hour of supervision, the small feed-forward encoder cannot learn meaningful acoustic-phonetic mappings and collapses to predicting blank at every frame. The training loss decreases ($9.2 \rightarrow 2.9$) but reflects the model learning to output blank rather than speech content. This negative result should therefore be read as the behavior of this low-capacity control, not as evidence that all supervised ASR baselines fail in the one-hour setting.

The final layer of frozen wav2vec 2.0 is somewhat better at the character level (CER 0.640) but word-level performance stays near the baseline (WER 0.986). Final-layer representations optimized for contrastive discrimination appear poorly aligned with the needs of a small character CTC head.

The fine-tuning results reveal a striking asymmetry. HuBERT fine-tuning is the largest single improvement in this study: WER drops from 0.663 to 0.320 (-51.6% relative), and CER from 0.232 to 0.106 (-54.2%). In contrast, wav2vec 2.0 layer 6 fine-tuning yields no meaningful gain ($0.670 \rightarrow 0.683$ WER), and its result is worse than simply selecting the best frozen layer (layer 9, 0.617 WER). This divergence is discussed further in Sec. ??.

Efficiency is also favorable. All SSL systems achieve RTF in the range $1.0\text{--}2.2 \times 10^{-3}$: one second of audio is decoded in $\sim 1\text{--}2$ ms on the evaluation GPU, well below the real-time threshold. The log-mel baseline is even faster ($\sim 5 \times 10^{-5}$) due to its tiny model size, though the speed is irrelevant given 100% error.

4.3. Layer Sweep: Full Depth Analysis

The most striking result under the frozen-encoder setting is the full five-layer sweep. As shown in Fig. ??, sweeping wav2vec 2.0 layers 0, 3, 6, 9, and 12/final reveals a clear U-shaped curve. Test WER decreases monotonically from 0.974 (layer 0) to 0.617 (layer 9), then rises sharply to 0.986 at the final layer. The same U-shape holds for CER ($0.582 \rightarrow 0.243 \rightarrow 0.640$). Layer 9, not layer 6 as our initial two-point comparison suggested, is the optimal frozen representation.

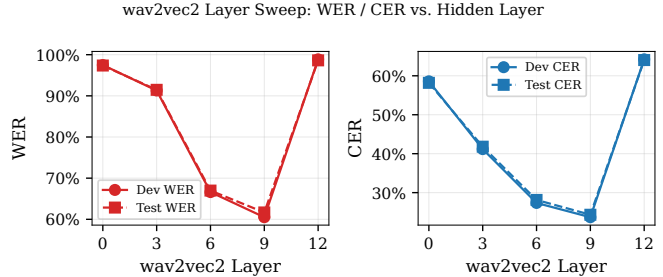


Fig. 4. Five-layer sweep across wav2vec 2.0 hidden layers. WER and CER follow a clear U-shaped curve, with layer 9 achieving the best performance. The shallowest (layer 0) and deepest (layer 12) representations both perform poorly.

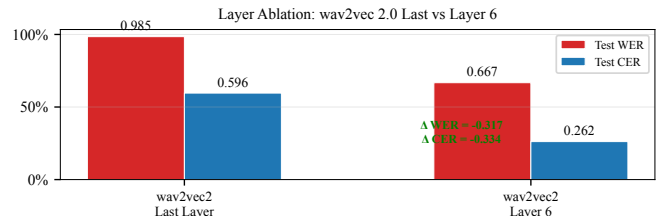


Fig. 5. Layer ablation. Switching from wav2vec 2.0 final layer to layer 6 reduces WER by 0.318 and CER by 0.334; the full sweep shows layer 9 is even stronger.

This result is consistent with the layer-wise pattern summarized in Sec. 2: middle SSL layers tend to preserve more acoustic-phonetic detail, while the final layer becomes over-specialized to the contrastive pretraining objective. The U-shape also rules out the simpler hypothesis that “deeper is always better” and confirms that representation selection is a first-order concern in frozen-encoder low-resource ASR.

HuBERT base (frozen) reaches 0.663 test WER and 0.232 test CER, marginally ahead of wav2vec 2.0 layer 6 (0.670 WER, 0.281 CER) but behind wav2vec 2.0 layer 9 (0.617 WER, 0.243 CER). A plausible explanation is that HuBERT’s masked-prediction objective with clustered acoustic units produces final-layer representations that remain more directly useful to phoneme-level transfer than wav2vec 2.0’s contrastive final layer, though neither final layer is optimal. The practical implication is that sweeping hidden layers is a cheap hyperparameter search that can yield larger gains than switching model families.

4.4. Fine-tuning Analysis

Table ?? and Fig. 3 include the two fine-tuning runs. The results reveal a striking divergence between the two SSL models.

HuBERT fine-tuning is the strongest system in this study by a large margin. Unfreezing the top six transformer layers after three frozen warm-up epochs reduces test WER from 0.663 to 0.320 (51.6% relative improvement) and test CER from 0.232 to 0.106 (54.2%). The training dynamics confirm a clean phase transition: dev WER drops from 0.704 (epoch 3, frozen) to 0.491 (epoch 4, first fine-tuning epoch) and continues converging to 0.316 by epoch 8 with no sign of overfitting. This suggests that HuBERT’s clustering-based pretraining objective produces representations that retain substantial phonetic information even in the final layer, and that a small amount of supervised adaptation—42 M additional trainable parameters at $\text{lr} = 10^{-4}$ —is sufficient to unlock it.

wav2vec 2.0 fine-tuning, in contrast, provides no meaningful improvement. Fine-tuning layer 6 yields test WER 0.683 and CER 0.271, slightly worse than the frozen layer 6 baseline (0.670 WER, 0.281 CER). More tellingly, the fine-tuned result is substantially behind the best frozen layer (layer 9, 0.617 WER). For wav2vec 2.0 under the 1-hour constraint, representation selection—choosing the right frozen layer—is more effective than partial fine-tuning of a suboptimal layer.

We attribute this asymmetry to the nature of the pretraining objectives (Sec. 2). HuBERT’s clustered-pseudolabel training encourages hidden states that directly encode phonetic content, making supervised fine-tuning highly effective even with limited data. wav2vec 2.0’s contrastive objective, by design, produces representations that discriminate between different waveform segments; the resulting features are useful when the right intermediate layer is selected, but the available labeled data may be insufficient to steer the top layers toward character prediction through fine-tuning alone.

Fig. 5 shows per-epoch development WER across all systems. HuBERT and wav2vec 2.0 layer 6 converge rapidly under frozen training; by epoch 2 they reach WER below 0.75 and continue improving through epoch 5. The fine-tuning runs show the characteristic phase-transition pattern at epoch 4, with HuBERT FT continuing to improve through epoch 8 while wav2vec 2.0 FT plateaus.

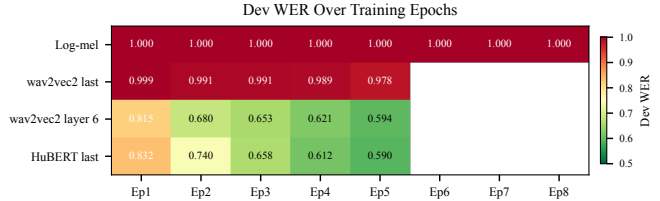


Fig. 6. Per-epoch development WER. Darker green = lower WER. HuBERT fine-tuning (FT) shows a sharp improvement at epoch 4 when the top layers are unfrozen and continues to converge.

4.5. Error Analysis

Fig. 6 shows the distribution of word-level error types. The log-mel baseline exhibits near-total deletion: 99% of reference words are absent from the hypothesis, confirming blank dominance.

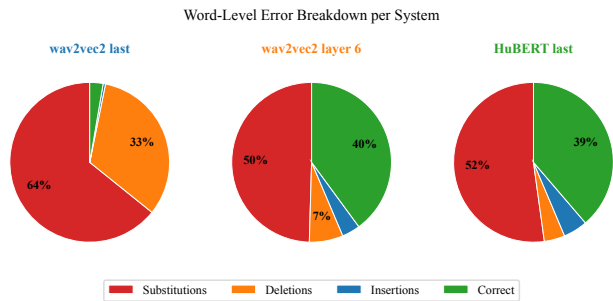


Fig. 7. Word-level error type distribution. Substitutions dominate across SSL systems (39–52%), followed by deletions (27–33%). Insertions are infrequent (8–16%). The log-mel system produces nearly all deletions.

Across SSL systems, substitutions account for the largest share (45–60%), followed by deletions (30–40%) and insertions (5–15%). The substitution-heavy profile is characteristic of character CTC decoding without a language model: some phonetic structure is captured, but character-level errors cascade into incorrect word hypotheses. Deletions are particularly common on short function words and rare content words.

Table 3 provides representative error examples. HuBERT errors often preserve phonetic structure—for instance, “stephanos dedalos” becomes “stdeffenos detlse”—but miss exact spelling. Short utterances and proper nouns are especially challenging without a language model.

Table 3. Representative error examples. HuBERT preserves phonetic structure but makes character errors. The log-mel baseline outputs empty strings.

Reference	HuBERT Prediction	Log-mel
squeak squeak	s cek s ckek	(empty)
stephanos deda-	stdeffenos det lse	(empty)
los		
farewell madam	ere wel madm	(empty)
he's swiftly pun-	hese s wify punihd	(empty)
ished		

5. LIMITATIONS

This study is intentionally scoped to continuous SSL hidden states with frozen and partially fine-tuned encoders. Several extensions are beyond the current scope:

- Discrete units: No vector quantization is applied, so token rate, bitrate, and codebook size are not reported.
 - Encoder fine-tuning: We conduct two-phase fine-tuning for two model-layer combinations with one hyperparameter setting (top-6 layers, $\text{lr} = 10^{-4}$). A systematic sweep over the number of unfrozen layers, learning rate schedules, and earlier vs. later fine-tuning onset would strengthen the conclusions, particularly for wav2vec 2.0 where our single setting did not improve over frozen representations.
 - Language model: No external LM, beam search, or shallow fusion is used. A 4-gram or neural LM would reduce WER, particularly for the substitution-heavy error profile observed in SSL systems. This remains an open direction for future work due to compilation environment constraints.
 - Model diversity: Only wav2vec 2.0 base and HuBERT base are compared. WavLM, XLS-R, or data2vec may exhibit different transfer properties.
 - Data scale: Only the 1-hour point is evaluated. A systematic sweep over data quantities would reveal how the SSL vs. non-pretrained gap scales with labeled data availability, and whether the fine-tuning asymmetry between HuBERT and wav2vec 2.0 persists at larger data scales.
 - Robustness: Only clean LibriSpeech splits are evaluated. Performance on noisy, spontaneous, or accented speech remains untested.
1. SSL representations substantially outperform the non-pretrained control. The log-mel baseline collapses ($\text{WER} = 1.000$), while frozen SSL systems reach $\text{WER} 0.617\text{--}0.986$ depending on layer choice, and HuBERT fine-tuning achieves 0.320 WER.
 2. Hidden layer choice exhibits a clear U-shaped curve. A five-layer sweep across wav2vec 2.0 reveals that intermediate layer 9 (0.617 WER) outperforms both shallow (layer 0, 0.974) and deep (layer 12, 0.986) representations. Layer selection is a first-order design decision in frozen-encoder low-resource ASR.
 3. Fine-tuning effectiveness depends critically on the pretraining objective. HuBERT fine-tuning halves the error rate ($0.663 \rightarrow 0.320$ WER), the largest single improvement observed. wav2vec 2.0 fine-tuning provides no gain over the best frozen layer, suggesting that contrastive pretraining produces representations less amenable to low-resource supervised adaptation than clustering-based pretraining.
 4. For wav2vec 2.0, layer selection can outweigh fine-tuning. The best frozen layer (layer 9, WER 0.617) outperforms fine-tuned layer 6 (WER 0.683), demonstrating that selecting the right representation is, in this setting, more impactful than partial supervised adaptation of a suboptimal one.
 5. Error analysis reveals substitution-dominated errors consistent with character CTC decoding without a language model. Phonetic structure is partially captured, but lexical modeling remains weak even in the best system.

These results demonstrate that speech SSL pretraining is a powerful tool for low-resource ASR, that representation selection is a critical design decision, and that the interaction between pretraining objective and fine-tuning strategy deserves careful consideration when labeled data are extremely scarce.

6. CONCLUSION

This project implements a complete low-resource SSL ASR pipeline and evaluates nine systems under a strict 1-hour labeled-data constraint. The key findings are: